

Universidade Federal de Ouro Preto - UFOP
Departamento de Computação - DECOM
BCC202 - Estruturas de Dados I

**Trabalho Prático II: Filas e Métodos de
Ordenação**
Simulação de Escalonador de Processos

Aluno: lucas Filipe Ramos

Matrícula: 20.14145

4 de fevereiro de 2026

Sumário

1	Implementação	2
1.1	Estruturas de Dados	2
1.2	Ordenação e Inserção (Insertion Sort)	2
1.3	Gerenciamento de Memória	3
2	Impressões Gerais	3
3	Análise	3
4	Conclusão	4

1 Implementação

O objetivo deste trabalho foi desenvolver um simulador de escalonador de processos para um sistema operacional, utilizando a linguagem C e o conceito de Tipo Abstrato de Dados (TAD) para gerenciar uma fila de prioridades[cite: 31, 33].

A implementação foi modularizada em três arquivos, conforme solicitado[cite: 76]:

- `tp.c`: Arquivo principal contendo a função `main`, responsável pela leitura dos dados e invocação do TAD.
- `filaprocessos.h`: Definição das estruturas e protótipos das funções.
- `filaprocessos.c`: Implementação da lógica de manipulação da fila e regras de negócio.

1.1 Estruturas de Dados

Para representar a fila, optou-se por uma **Lista Encadeada Simples**. A escolha se deu pela dinamicidade da estrutura, permitindo a inserção e remoção de processos sem a necessidade de realocação contígua de memória, o que é ideal para um cenário onde processos são constantemente removidos do início e reinseridos no meio ou fim da fila.

A estrutura do processo armazena o ID, tempo de chegada, prioridade, ciclos totais e ciclos já executados. O TAD `FilaProcessos` mantém ponteiros para o início e fim da lista, além de um vetor dinâmico para armazenar o log de execução.

1.2 Ordenação e Inserção (Insertion Sort)

Conforme as regras do trabalho, foi utilizado o algoritmo **Insertion Sort** para manter a fila ordenada[cite: 69].

Houve uma distinção crucial na lógica de ordenação identificada durante os testes:

1. **Carga Inicial:** A ordenação primária é feita pela **Prioridade** (menor valor = maior prioridade). Em caso de empate na prioridade, o desempate é feito pelo **Tempo de Chegada** (quem chegou antes, executa antes). Essa decisão foi tomada após análise dos casos de teste (especificamente `teste3` e `teste4`), onde a ordenação puramente por tempo de chegada falhava.
2. **Reinserção:** Quando um processo excede o quantum ($X = 100$)[cite: 36], ele é penalizado (prioridade incrementada) e reinserido. Neste caso, a ordenação considera apenas a **Prioridade**. Para garantir a estabilidade do algoritmo, processos com mesma prioridade são inseridos após os já existentes.

Trecho da função de inserção corrigida:

```
1 // Trecho simplificado da logica de insercao
2 while (atual->prox != NULL) {
3     int p_prox = atual->prox->proc.prioridade;
4     float t_prox = atual->prox->proc.tempo_chegada;
5
6     // Avanca se o proximo for mais prioritario
7     if (p_prox < novo->proc.prioridade) {
8         atual = atual->prox;
9     }
```

```

10 // Avança se tiver mesma prioridade mas chegou antes
11 else if (p_prox == novo->proc.prioridade &&
12         t_prox <= novo->proc.tempo_chegada) {
13     atual = atual->prox;
14 }
15 else {
16     break;
17 }
18 }

```

Listing 1: Lógica de inserção ordenada

1.3 Gerenciamento de Memória

Para evitar *memory leaks*[cite: 39], todas as estruturas alocadas com `malloc` (nós da lista e vetores de log) são liberadas na função `destroiFila`. Foi utilizada a ferramenta **Valgrind** para validação, garantindo que não houvesse vazamentos ou acessos indevidos à memória[cite: 40].

2 Impressões Gerais

O desenvolvimento deste trabalho foi extremamente enriquecedor para consolidar o entendimento sobre manipulação de ponteiros e listas encadeadas. O que mais gostei foi a aplicação prática de um conceito teórico (filas) em um problema real de computação (escalonamento de CPU). Ver o log de saída bater exatamente com os arquivos de teste fornecidos gerou uma grande satisfação.

A parte que me desagradou levemente foi a complexidade inicial de depurar erros de segmentação (*segmentation faults*) quando os ponteiros da lista não eram atualizados corretamente nas reinserções. No entanto, isso me forçou a utilizar `gdb` e `valgrind`, ferramentas essenciais para um cientista da computação.

3 Análise

A complexidade do algoritmo gira em torno da operação de inserção. Como utilizamos uma lista encadeada ordenada (Insertion Sort):

- **Melhor caso:** $O(1)$, quando o processo deve ser inserido no início da fila.
- **Pior caso:** $O(N)$, quando o processo tem a menor prioridade e deve ir para o final.
- **Complexidade Total:** Para N processos com múltiplas reinserções, a complexidade tende a $O(N^2)$ no pior cenário.

Durante a execução dos casos de teste, observou-se que a lógica de desempate é fundamental. Inicialmente, tentei ordenar apenas por tempo de chegada, mas isso gerou divergências nos testes onde processos chegavam no tempo 0.00 com prioridades diferentes. A correção para priorizar a prioridade (valha a redundância) sobre o tempo foi o ponto chave para a corretude do algoritmo.

4 Conclusão

O Trabalho Prático II permitiu compreender profundamente o funcionamento de filas de prioridade e a importância da gestão manual de memória em C. As principais dificuldades encontradas foram relacionadas à lógica de ordenação composta (Prioridade + Tempo) e ao tratamento de casos de borda (fila vazia ou inserção no início).

A implementação modularizada facilitou os testes e a detecção de erros. Concluiu-se que o uso de TADs é vital para manter o código organizado e que a validação com casos de teste rigorosos é indispensável para garantir a robustez de sistemas críticos como um escalonador.